

AprioriAllPkg: A Python package for mining sequential patterns

Aleksander Kruczkowski^a, Pawel Weichbroth^{b,*}

^a Capgemini, Gdansk, Poland

^b Faculty of Electronics, Telecommunications and Informatics, Department of Software Engineering, Gdansk University of Technology, Gdansk, Poland

ARTICLE INFO

Keywords:

Frequent
Pattern
Sequence
Mining
AprioriAll

ABSTRACT

This work presents a software package designed to support transparency, education, and methodological reproducibility in sequential pattern mining. The package concerns the AprioriAll algorithm, a classical method for discovering frequent ordered patterns in transactional data. Despite its historical importance, accessible and faithful implementations of AprioriAll remain difficult to find in modern software ecosystems, where available tools are often written in other programming languages or focused on newer and more complex approaches. The proposed package addresses this gap through a native Python implementation that is lightweight, modular, and independent of external libraries. Its structure follows the original stages of the AprioriAll mining process, allowing users to inspect, execute, and modify each component of the algorithm independently. This enables researchers and practitioners to reproduce complete mining workflows, test alternative configurations, and extend selected stages while preserving interpretability. By exposing the internal mechanics of candidate generation, support counting, pruning, and sequence expansion, the package provides a practical environment for understanding the computational challenges of temporal data mining. At the same time, it offers a convenient foundation for benchmarking, methodological validation, performance evaluation, and rapid prototyping of sequential data analysis workflows.

Metadata

Software metadata.

Nr.	Software metadata description	Metadata
S1	Software name	AprioriAllPkg
S2	Software version	v1.0
S3	Programming language	Python
S4	Operating system compatibility	Platform independent; tested with Python 3.10 +
S5	Main functionality	Python implementation of the AprioriAll algorithm for mining frequent and maximal sequential patterns from transactional sequence data

Code metadata.

Nr.	Code metadata description	Metadata
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	https://github.com/AleksanderKr/AprioriAllPkg
C3	Permanent link to Reproducible Capsule	https://mybinder.org/v2/gh/AleksanderKr/AprioriAllPkg/HEAD?urlpath=%2Fdoc%2Ftree%2FAprioriAllDemo.ipynb
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python
C7	Compilation requirements, operating environments & dependencies	Python 3.10 +
C8	If available Link to developer documentation/manual	https://github.com/AleksanderKr/AprioriAllPkg/blob/main/README.md
C9	Support email for questions	kruczkowski.aleksander@gmail.com

* Corresponding author.

Email addresses: kruczkowski.aleksander@gmail.com (A. Kruczkowski), pawel.weichbroth@pg.edu.pl (P. Weichbroth).

1. Motivation and significance

Sequential pattern mining is a fundamental task in data science [1]. While contemporary research often prioritizes scalable, high-performance sequential pattern mining approaches such as PrefixSpan [2] and SPAM [3], the structured logic of the AprioriAll algorithm, introduced by Agrawal and Srikant [4], serves as a primary benchmark and reference model [5]. It is widely recognized as one of the foundational algorithms in sequential pattern mining and remains an important reference point for understanding the computational challenges of discovering frequent ordered patterns in large transactional datasets [6], while explicitly illustrating the difficulties of temporal data processing in a way that more opaque or complex systems often do not [7].

However, a noticeable gap remains in the modern software landscape. Most comprehensive libraries for sequential pattern mining, such as SPMF [8], are implemented primarily in Java or are focused on newer and more complex algorithms. For researchers and practitioners working within the Python ecosystem, finding a transparent, readable, and faithful implementation of the original AprioriAll algorithm remains difficult.

Although several sequential pattern mining implementations are available within Python ecosystems and public repositories, many of them focus on specific optimized algorithms, provide limited transparency into intermediate processing stages, or rely on external mining frameworks. In contrast, the presented package emphasizes a lightweight, modular, and reproducible implementation that closely follows the original AprioriAll workflow and exposes its individual processing stages for inspection and experimentation.

The main significance of the *AprioriAllPkg* software package lies in its ability to address this gap by providing a reference implementation of the AprioriAll algorithm. By offering a Python tool that is independent of external libraries, the package allows users to examine each phase of the algorithm in detail. This facilitates a deeper understanding of the classical mining pipeline. It also provides a lightweight foundation for methodological validation, performance evaluation, scalability analysis, benchmarking, and rapid prototyping of sequential data workflows.

2. Software description

The software provides a standalone Python implementation of the AprioriAll algorithm designed to work with transactional data in long format text-based files.

2.1. Software architecture

The architecture is built around a linear data processing pipeline that strictly mirrors the five logical steps of the AprioriAll algorithm. The initial data ingestion and preparation phase (Step 1) is illustrated in Fig. 1, while the execution of the mining workflow (Steps 2–5) is presented in Fig. 2.

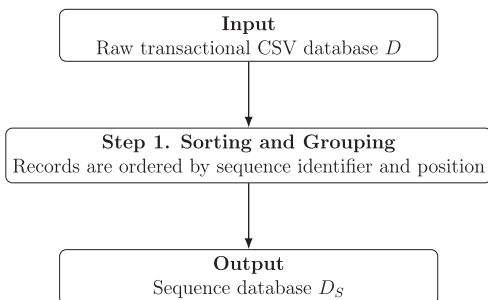


Fig. 1. Data preprocessing pipeline corresponding to Step 1 of the AprioriAll algorithm.

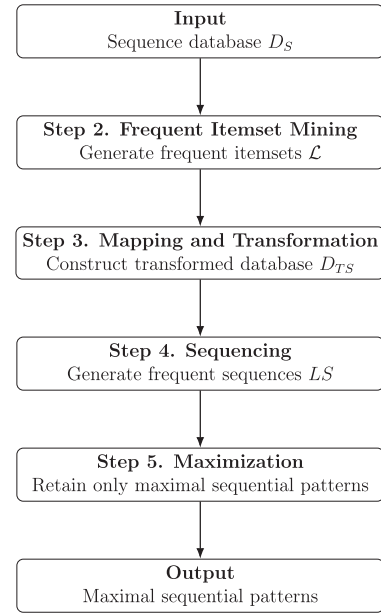


Fig. 2. Internal workflow of the AprioriAll mining engine.

- **Sorting and Grouping (Step 1):** The software reads raw transactional data from database D and groups records chronologically by sequence identifiers into database D_S . As shown in the illustrative example (see Fig. 1), flat records for a single entity are merged into a single sequence structure $\langle(30)(90)\rangle$.
- **Mining Frequent Sets (Step 2):** The system identifies all frequent itemsets across the dataset utilizing subset checks and a level-by-level search to produce frequent itemsets L .
- **Transformation (Step 3):** The system maps discovered frequent itemsets to unique integer identifiers. The sequence database is then transformed into sequences of mapped integers (D_{TS}), simplifying the subsequent sequence discovery phase.
- **Sequencing (Step 4):** The core engine uses candidate generation to find frequent sequences (L_S). It generates candidates of length k from frequent sequences at level $k - 1$, evaluates support via the transformed database, and prunes invalid patterns.
- **Maximization (Step 5):** The final component filters the results to retain only maximal sequences, removing sub-patterns fully contained within longer frequent sequences (see Fig. 2).

2.2. Software functionalities

The package exposes its core logic through key functions that map directly to the architecture described above:

- `read_sequences_long_itemsets(path)`: Handles data ingestion for Step 1. It parses CSV files and groups items into ordered lists of frozenset objects based on identifiers and positions.
- `mine_frequent_itemsets(sequences, min_sup_count)`: Executes Step 2 by identifying all itemsets meeting the minimum support threshold.
- `apriori_all(sequences, min_sup_count)`: The main engine orchestrating the entire workflow. It handles integer mapping, database transformation (Step 3), candidate evaluation (Step 4), and maximization.
- `filter_maximal_sequences(seq_counts)`: A subsequent utility for Step 5 that filters the discovered frequent sequences to return only maximal patterns.

```

1 while prev_ls:
2     ck = apriori_generate_sequences(prev_ls, k)
3
4     if not ck:
5         break
6
7     counts = count_sequence_support(ck, transformed_db)
8     current_ls = {cand: c for cand, c in counts.items() if c
9                   >= min_sup_count}
10
11    if not current_ls:
12        break
13
14    all_mapped_sequences.update(current_ls)
15    prev_ls = set(current_ls.keys())
16    k += 1

```

Listing 1. Core sequencing loop (Step 4).

```

1 def apriori_generate_sequences(prev_ls, k):
2     prev_list = list(prev_ls)
3     candidates = set()
4
5     for i in range(len(prev_list)):
6         for j in range(len(prev_list)):
7             a, b = prev_list[i], prev_list[j]
8             if a[:-1] == b[:-1]:
9                 cand = a + (b[-1],)
10
11                valid = True
12                for drop in range(k):
13                    sub = cand[:drop] + cand[drop + 1:]
14                    if sub not in prev_ls:
15                        valid = False
16                        break
17
18                if valid:
19                    candidates.add(cand)
20
21    return candidates

```

Listing 2. Candidate generation with Join and Prune steps.

2.3. Code snippets

The implementation logic is illustrated through two critical algorithmic phases. Listing 1 demonstrates the central execution loop of the algorithm corresponding to Step 4. The logic orchestrates the level-by-level search, evaluates candidate support against the transformed database, and updates the pool of valid patterns.

The candidate generation phase is presented in Listing 2. This function combines pairs of frequent sequences sharing an identical prefix (join step) and verifies whether all $(k - 1)$ -subsequences of the new candidate exist in the previously discovered frequent set (prune step).

2.4. Installation

The software requires Python 3.10 or higher. The package is executed directly from the source code. The toolkit is installed by cloning the public repository:

```
git clone https://github.com/AleksanderKr/AprioriAllPkg.git
```

For a detailed installation and usage guide, please refer to the repository's main page.

The package natively processes two distinct data configurations:

- **CSV Format:** The input CSV file must contain three columns: `sequence_id`, `pos`, and `item`. The `sequence_id` identifies a

sequence, `item` represents an observed item, and `pos` specifies the position (order) of the corresponding element within the sequence. Items sharing the same `pos` value within a sequence are grouped into a single itemset.

- **SPMF Format:** The package contains a parser for standard `.txt` files compliant with the SPMF format. A standalone conversion utility (`utils/convert_spmf_to_seq.py`) is provided for bidirectional formatting.

3. Illustrative examples

Table 1 presents a sample of a database D from a medical clinic containing four attributes: a timestamp (*Date*), a patient identifier (*PatientID*), a prescription identifier (*PrescriptionID*), and the assigned drug (*DrugID*). Given a minimum support threshold (*minsup*) of 0.25, identify the frequent sequential patterns and the maximal sequential patterns.

The AprioriAll algorithm consists of five steps.

Step 1. Sorting and grouping

The database D should first be sorted by the *PatientID* attribute and subsequently by the *Date* attribute. In the next step, the data are grouped according to the *PatientID* attribute. The outcome of these two operations is shown in Table 2.

Table 1
Subset of medical clinic database.

Date	PatientID	PrescriptionID	DrugID
01/26/98	105	112	30
01/26/98	106	113	10
01/26/98	106	113	20
02/02/98	105	114	90
02/10/98	106	115	30
03/08/98	106	116	40
03/08/98	106	116	60
03/08/98	106	116	70
03/09/98	200	117	30
03/09/98	200	117	50
03/09/98	200	117	70
03/09/98	220	118	30
03/10/98	300	119	90
03/15/98	220	120	40
03/15/98	220	120	70
03/18/98	220	121	90

Table 2
Grouped drug sequences by patient (D_S sequence database).

PatientID	DrugID
105	$\langle(30) (90)\rangle$
106	$\langle(10, 20) (30) (40, 60, 70)\rangle$
200	$\langle(30, 50, 70)\rangle$
220	$\langle(30) (40, 70) (90)\rangle$
300	$\langle(90)\rangle$

Note that the D_S database contains five sequences, comprising six 1-itemsets, two 2-itemsets, and two 3-itemsets, where the elements within each itemset are ordered in ascending order.

Step 2. Mining frequent sets

Note that to extract frequent itemsets from the database D , any frequent itemset mining algorithm can be used. In this example, we apply the Apriori algorithm, as it is one of the most widely recognized methods to date.

In the first step, the Apriori algorithm generates a 1-itemset candidate set $C_1 = \{(10), (20), (30), (40), (50), (60), (70), (90)\}$. Next, it computes the support for each candidate itemset: $(30) : 0.80, (40) : 0.40, (70) : 0.60, (90) : 0.60$. Thus, 1-item frequent set is $L_1 = \{(30), (40), (70), (90)\}$, and therefore $|L_1| = 4$.

Next, we generate the set $C_2 = \{(30, 40), (30, 70), (30, 90), (40, 70), (40, 90), (70, 90)\}$, hence $|C_2| = 6$. After computing the support for each itemset, we obtain the following results: $(30, 40) : 0, (30, 70) : 0.20, (30, 90) : 0, (40, 70) : 0.40, (40, 90) : 0, (70, 90) : 0$. Thus, $L_2 = \{(40, 70)\}$ and therefore $|L_2| = 1$.

Next, the set C_3 is generated. The first and only possible 3-itemset that can be produced is $(40, 70, 90)$. However, its 2-item subset $(70, 90)$ is not frequent. Therefore, $C_3 = \emptyset$, which means that the algorithm terminates.

The result of the Apriori algorithm is the family of frequent itemsets $\mathcal{L} = \{L_1, L_2\}$.

Step 3. Transformation

In the third step, the sequence database D_S is transformed into an ordered list of itemsets D_{TS} . First, each frequent item is mapped to a corresponding unique natural number, as shown in Table 3.

Second, using the above mapping, each element T of a sequence S is replaced by an ordered list of frequent itemsets contained in T . Consequently, all items in T that are not frequent are removed, as they cannot form part of a frequent sequential pattern.

Table 3
Frequent itemsets and their corresponding mappings to natural numbers.

Frequent itemset	Support	Mapping
(30)	0.80	1
(40)	0.40	2
(70)	0.60	3
(40, 70)	0.40	4
(90)	0.60	5

Table 4
Transformation and mapping of sequences.

Original sequence	After transformation	After mapping
$\langle(30) (90)\rangle$	$\langle((30)) ((90))\rangle$	$\langle(1) (5)\rangle$
$\langle(10, 20) (30) (40, 60, 70)\rangle$	$\langle((30)) ((40), (70), (40, 70))\rangle$	$\langle(1) (2\ 3\ 4)\rangle$
$\langle(30, 50, 70)\rangle$	$\langle((30), (70))\rangle$	$\langle(1, 3)\rangle$
$\langle(30) (40, 70) (90)\rangle$	$\langle((30)) ((40), (70), (40, 70)) ((90))\rangle$	$\langle(1) (2\ 3\ 4) (5)\rangle$
$\langle(90)\rangle$	$\langle((90))\rangle$	$\langle(5)\rangle$

Note that if a sequence does not contain any frequent itemsets, it is not included in the transformed database D_{TS} ; however, it is still taken into account when computing the support of a sequential pattern (Table 4).

Consider the second sequence in the form $\langle(10, 20), (30), (40, 60, 70)\rangle$. Initially, it contains three elements: one 2-itemset $(10, 20)$, followed by one 1-itemset (30) , and one 3-itemset $(40, 60, 70)$. First, the infrequent items, namely 10, 20, and 60, are removed. As a result, the sequence becomes $\langle(30), (40, 70)\rangle$, which consists of two elements: one 1-itemset (30) and one 2-itemset $(40, 70)$.

Next, the sequence is transformed such that each of its elements is replaced by an ordered list of frequent itemsets. Consequently, the itemset $(40, 70)$ is decomposed into two 1-itemsets and one 2-itemset. Finally, after the transformation, the sequence takes the form $\langle((30)) ((40), (70), (40, 70))\rangle$, and after mapping it becomes $\langle(1) (2\ 3\ 4)\rangle$.

Step 4. Sequencing

The set of frequent sequences, denoted by LS_1 , is the result of the frequent itemset discovery step. In the analyzed case, $L_1 = \{(30), (40), (70), (90)\}$ and $L_2 = \{(40, 70)\}$. Consequently, after mapping (see Table 3), the set $LS_1 = \{1, 2, 3, 4, 5\}$. Note that the mapped values represent frequent itemsets rather than individual drugs. For instance, the item 4 corresponds to the frequent itemset $(40, 70)$.

To discover subsequent frequent sequences, the function `apriori_generate()` is called. Based on the frequent itemsets, frequent sequences (i.e., sequential patterns) are generated according to the AprioriAll algorithm. Let LS_k denote the set of frequent sequences of length k , and CS_k the set of candidate sequences of length k . The candidate k -sequences are generated from the $(k-1)$ frequent sequences in two stages of the `apriori_generate()` function:

- **Join step:** Two sequences from the set LS_{k-1} are combined to generate a candidate k -sequence. The condition for joining two sequences from LS_{k-1} is that the first $(k-2)$ elements of both sequences are identical.
- **Prune step:** All sequences $S \in CS_k$ are removed if there exists a $(k-1)$ -subsequence of S that does not belong to LS_{k-1} .

From LS_1 , the set of candidate 2-sequences CS_2 is generated, for which the support is subsequently computed (see Table 5).

Thus, the cardinality of the set of candidate 2-sequences is equal to 25. Considering the minimum support threshold, only four sequences satisfy the required support, namely $\langle 1\ 2 \rangle$, $\langle 1\ 3 \rangle$, $\langle 1\ 4 \rangle$, and $\langle 1\ 5 \rangle$. Therefore, $|LS_2| = 4$. In the next step, the function `apriori_generate()` produces the following set of candidate sequences during the join stage:

Table 5
Candidate 2-sequences and their support values.

Sequence	Support
(1 1)	0
(1 2)	0.40
(1 3)	0.40
(1 4)	0.40
(1 5)	0.40
(2 1)	0
(2 2)	0
(2 3)	0
(2 4)	0
(2 5)	0.20
(3 1)	0
(3 2)	0
(3 3)	0
(3 4)	0
(3 5)	0.20
(4 1)	0
(4 2)	0
(4 3)	0
(4 4)	0
(4 5)	0
(5 1)	0
(5 2)	0
(5 3)	0
(5 4)	0
(5 5)	0

(1 2 2), (1 2 3), (1 2 4), (1 2 5),
 (1 3 2), (1 3 3), (1 3 4), (1 3 5),
 (1 4 2), (1 4 3), (1 4 4), (1 4 5),
 (1 5 2), (1 5 3), (1 5 4), (1 5 5).

The set of candidate 3-sequences CS_3 is empty, since each of the above sequences contains a 2-subsequence that does not belong to the set LS_2 . Therefore, the AprioriAll algorithm terminates.

Step 5. Maximization (optional)

In this step, the sequences in LS_2 that are not maximal are removed. The sequences (1 2) and (1 3) are not maximal, since they are contained in the sequence (1 4). This follows from the fact that the itemsets (30) and (40) correspond to, respectively, the first element and the first item of the second element of the fourth sequence, while the itemset (70) corresponds to the second item of the second element of this sequence.

Ultimately, the set of maximal sequential patterns for $minsup = 0.25$ consists of two patterns:

$\langle(30) (90)\rangle : 0.4$ and $\langle(30) (40, 70)\rangle : 0.4$.

4. Experimental results and performance evaluation

4.1. Experimental setup

To ensure reproducibility and provide a clear baseline for the reported execution metrics, all benchmarks were conducted on a dedicated local workstation. The detailed hardware and software specifications of the experimental environment are summarized in Table 6. All scripts, algorithmic evaluations, and performance profiling were executed entirely on the CPU utilizing a single-threaded approach, except where explicitly noted as parallelized. Note that CPU utilization was measured using the `psutil` library and corresponds to the average CPU usage of the benchmarked process during execution.

Execution times represent the average of multiple independent runs. Standard deviations were omitted from the reported tables, as

Table 6
Hardware and software specifications of the experimental environment.

Component	Specification
CPU	AMD Ryzen 7 7700X
RAM	32 GB (DDR5, 6000 MHz)
GPU	NVIDIA GeForce RTX 4070 SUPER
OS	Windows 11 Pro (Version 25H2, 64-bit)
Environment	Python 3.12.7

the minor variance across individual runs is negligible compared to the orders-of-magnitude performance difference between the Python implementation and the JVM-based reference algorithms.

4.2. Minimal working examples and tracing

To utilize the modules directly within Python scripts, users instantiate a programmatic database and invoke the mining components. Listing 3 demonstrates sequence construction, algorithm initialization, and pattern extraction executed from the repository root.

Command-line execution and trace. To verify that the software correctly implements the theoretical framework described in Section 3, the package provides a `--debug` flag for direct command-line execution. While the unit tests validate the output automatically, this flag forces the algorithm to explicitly print its internal state at each structural step of the process.

Note that the `--debug` flag introduces significant standard output overhead and is intended strictly for educational verification on small, sample datasets. It should not be used when processing large, real-world databases.

Users can reproduce the theoretical workflow using the dataset corresponding to Table 2. Setting the core parameter `--min-sup-count` to 2 yields the command shown in Listing 4.

The software generates a step-by-step console execution log mapping directly to the structural execution layers of the algorithm (with candidate sequence generation omitted for brevity). This terminal trace, presented in Listing 5, mirrors the manual calculations, proving the structural integrity of the implementation.

Output interpretation. Upon completing execution, the software writes the discovered patterns to three CSV files stored in the output directory. The generated files provide complementary views of the mining results and facilitate both human interpretation and subsequent computational analysis:

- `maximal_sequences_human.csv`: Contains maximal sequential patterns represented using the original item labels together with their support values.
- `maximal_sequences.csv`: Contains the same maximal sequential patterns encoded using the internal integer mapping generated during the transformation stage.
- `frequent_sequences.csv`: Contains the complete set of discovered frequent sequential patterns represented using the internal integer encoding.

Together, these files provide access to both the complete set of frequent patterns and the reduced set of maximal patterns, enabling reproducibility, further analysis, and integration with external data processing workflows.

Reproducibility. To facilitate full reproducibility beyond local execution, the repository is linked to a reproducible capsule hosted on Binder.


```

1 from src.apriori_all import AprioriAll
2 from src.data_structures import SequenceDatabase
3
4 db = SequenceDatabase()
5
6 db.add_sequence(1, [(10, 'A'), (10, 'B'), (20, 'C'), (30, 'D')
7 ])
8 db.add_sequence(2, [(10, 'A'), (20, 'D'), (20, 'E')])
9 db.add_sequence(3, [(10, 'A'), (10, 'B'), (20, 'D')])
10 db.add_sequence(4, [(10, 'B'), (20, 'C'), (30, 'D')])
11
12 miner = AprioriAll(min_sup_count=2)
13 maximal_patterns = miner.mine(db)
14
15 for pattern, support in maximal_patterns:
16     print(f"Pattern: {pattern} | Support: {support}")

```

Listing 3. Minimal working example using the Python codebase.

```

1 python src/apriori_all.py --sequences data/sequences_test.txt
   --min-sup-count 2 --debug

```

Listing 4. Command line execution with the debug verification flag.

4.3. Algorithmic validation and reproducibility

To ensure the reliability of *AprioriAllPkg*, validation was performed through automated unit testing, empirical cross-validation against established sequential pattern mining algorithms, and manual execution tracing. The repository includes automated unit tests covering the core software components and illustrative examples.

To assess correctness on larger datasets, we conducted a comprehensive benchmark study comparing the Python implementation of *AprioriAll* (both single-threaded and parallelized) against established sequential pattern mining algorithms available in the SPMF framework [8], namely PrefixSpan [2], SPADE [9], and GSP.

For each dataset and minimum support threshold, all algorithms were configured to return the complete set of frequent sequential patterns. Correctness was verified by comparing the generated pattern sets across all evaluated methods. The resulting pattern sets were compared for equality, and a subset of the outputs was manually inspected to verify the consistency of support values and pattern representations.

As shown in Table 7, *AprioriAllPkg* consistently produced identical results to the reference implementations across all tested configurations. For example, when processing the *Sign* dataset with a minimum support count of 80, all evaluated algorithms discovered exactly 81,340 frequent sequential patterns, while a minimum support count of 120 yielded 20,024 patterns. The observed agreement in both pattern counts and pattern sets confirms the algorithmic correctness of the implementation and demonstrates that the complete *AprioriAll* workflow, from candidate generation to maximal sequence filtering, is faithfully reproduced.

To facilitate independent verification and full reproducibility, the repository includes automated test scripts, benchmark datasets, and a Jupyter notebook for reproducing the reported validation results and performance measurements.

4.4. Performance and scalability analysis

Experiments were conducted using five standard datasets characterized by varying sequence lengths, densities, and dimensions: *Sign*, *Leviathan*, *Bike*, *Bible*, and *BMSWebView1*. Table 7 summarizes the execution time, peak memory footprint, and CPU utilization across different support thresholds. The parallel implementation distributes candidate support counting across *N* worker processes using Python multiprocessing while preserving identical algorithmic behavior.

The empirical evaluation highlights several key behavioral characteristics of the algorithms:

- **Execution Time and Dataset Density:** As an uncompiled Python implementation, *AprioriAll* exhibits significantly lower overall performance and higher execution times than specialized, JVM-based mining frameworks. This is particularly evident in dense datasets with long pattern sequences, such as *Sign* (*minsup* = 80), where *AprioriAll* required approximately 751 s. PrefixSpan consistently achieved sub-second execution times across most tests by relying on pattern-growth methodologies, which eliminate the computationally expensive candidate generation phase entirely.
- **Memory Overhead and Resource Utilization:** While the algorithm yields lower execution performance, the single-threaded implementation remains relatively resource-efficient in terms of base memory utilization (peaking at 435 MB for the *Bible* dataset). The primary limitation of the *AprioriAll* approach is its memory footprint scaling during the horizontal dataset scanning and in-memory candidate maintenance, though modern SPMF algorithms are not completely exempt from high memory demands either.
- **Parallelization:** We included a parallelized variant of *AprioriAll* (*AprioriAll (Par.)*) primarily as an educational demonstration of the algorithm's modularity. While it successfully leverages multi-core architectures to reduce execution time (e.g., from 1592 s to roughly 150 s for *BMSWebView1*), this comes at the cost of duplicating candidate structures across child processes, increasing memory usage nearly tenfold. This highlights that while parallelization can offer raw speedups, fundamental algorithmic improvements (such as those introduced by GSP, SPADE, or PrefixSpan) are required for true scalability in production environments.

In conclusion, the benchmark suite demonstrates that while *AprioriAllPkg* is not designed to surpass modern projection-based or vertical-format architectures in terms of computational performance, it is a fully functional and algorithmically correct implementation. It successfully fulfills its intended purpose as a transparent and reproducible Python package for educational use, methodological validation, and experimental research in sequential pattern mining.

5. Impact

By providing a Python-based implementation of the *AprioriAll* algorithm, this package addresses the limited availability of open

```

1  === RUNNING APRIORI ALL ALGORITHM ===
2  Total sequences loaded: 5
3  Minimum Support Count threshold: 2 (40.00)
4
5  === Step 1: Sorted and Grouped Sequences Database (Ds) (Total
   : 5) ===
6  <{30},{90}>
7  <{10,20},{30},{40,60,70}>
8  <{30,50,70}>
9  <{30},{40,70},{90}>
10 <{90}>
11
12 === Step 2: Discovered Frequent Itemsets (L) (Total: 5) ===
13 {30}
14 {40}
15 {70}
16 {90}
17 {40,70}
18
19 === Step 3: Frequent Itemsets Mapping to Integers (Total: 5)
   ===
20 {30} -> 1
21 {40} -> 2
22 {70} -> 3
23 {90} -> 4
24 {40,70} -> 5
25
26 === Step 3: Transformed and Mapped Database (Dts) (Total: 5)
   ===
27 <{1},{4}>
28 <{1},{2,3,5}>
29 <{1,3}>
30 <{1},{2,3,5},{4}>
31 <{4}>
32
33 === Step 4: Frequent 1-Sequences (LS1) (Total: 5) ===
34 <1> (sup: 4)
35 <2> (sup: 2)
36 <3> (sup: 3)
37 <4> (sup: 3)
38 <5> (sup: 2)
39
40 === Step 4: Frequent 2-Sequences (LS2) (Total: 4) ===
41 <1,2> (sup: 2)
42 <1,3> (sup: 2)
43 <1,4> (sup: 2)
44 <1,5> (sup: 2)
45
46 === Step 5: All Frequent Sequential Patterns Before
   Maximization (Total: 9) ===
47 <{30}> (sup: 4)
48 <{40}> (sup: 2)
49 <{40,70}> (sup: 2)
50 <{70}> (sup: 3)
51 <{90}> (sup: 3)
52 <{30},{40}> (sup: 2)
53 <{30},{40,70}> (sup: 2)
54 <{30},{70}> (sup: 2)
55 <{30},{90}> (sup: 2)
56
57 === Step 5: Final Maximal Sequential Patterns (Total: 2) ===
58 <{30},{40,70}> (sup: 2)
59 <{30},{90}> (sup: 2)
60
61 OK: Execution finished. Discovered 9 ALL frequent patterns, 2
   MAXIMAL sequential patterns.

```

Listing 5. Software-generated execution trace showing internal algorithmic states.

Table 7

Performance evaluation across benchmark datasets.

Dataset	Sequence Count	Minimum Support Count	Frequent Patterns	Algorithm	Execution Time (s)	Peak Memory (MB)	CPU Usage (%)
Sign	730	80	81,340	AprioriAll	751.59	72.26	6.4
				AprioriAll (Par.)	450.13	465.82	10.4
				PrefixSpan	1.62	547.16	9.0
				SPADE	1.73	693.02	12.0
				GSP	24.65	288.02	6.9
	120	20,024	20,024	AprioriAll	116.55	39.54	6.4
				AprioriAll (Par.)	45.17	465.65	15.2
				PrefixSpan	0.77	537.78	9.4
				SPADE	0.77	667.00	17.8
				GSP	6.46	242.47	7.3
	160	6775	6775	AprioriAll	43.50	34.53	6.3
				AprioriAll (Par.)	16.21	464.58	16.1
				PrefixSpan	0.50	215.28	11.7
				SPADE	0.55	296.07	18.7
				GSP	2.97	223.21	8.3
Leviathan	5834	583	651	AprioriAll	27.68	111.62	6.4
				AprioriAll (Par.)	13.91	1227.78	14.0
				PrefixSpan	0.39	161.40	10.9
				SPADE	0.61	537.41	19.5
				GSP	2.71	559.85	8.3
	875	226	226	AprioriAll	8.44	111.61	6.4
				AprioriAll (Par.)	9.61	1245.70	9.7
				PrefixSpan	0.33	119.64	12.7
				SPADE	0.56	538.55	19.0
				GSP	1.11	434.52	10.7
	1167	101	101	AprioriAll	4.71	111.09	6.3
				AprioriAll (Par.)	8.84	1227.48	8.3
				PrefixSpan	0.34	109.86	19.9
				SPADE	0.50	525.04	17.4
				GSP	0.72	505.32	10.8
Bike	21,078	210	907	AprioriAll	191.36	98.90	6.4
				AprioriAll (Par.)	29.27	1072.53	50.4
				PrefixSpan	0.34	116.68	13.3
				SPADE	0.67	589.58	14.1
				GSP	6.32	239.16	7.1
	420	263	263	AprioriAll	61.24	97.57	6.4
				AprioriAll (Par.)	14.31	1065.14	31.5
				PrefixSpan	0.34	124.12	14.2
				SPADE	0.45	287.50	12.2
				GSP	5.71	238.73	6.8
	840	64	64	AprioriAll	34.30	97.45	6.4
				AprioriAll (Par.)	10.33	1064.43	25.2
				PrefixSpan	0.28	116.50	16.6
				SPADE	0.33	203.38	13.8
				GSP	3.87	238.48	7.2
Bible	36,369	1818	774	AprioriAll	165.25	435.72	6.4
				AprioriAll (Par.)	42.81	4195.16	30.5
				PrefixSpan	0.77	401.45	10.9
				SPADE	2.49	5189.47	16.6
				GSP	14.40	587.21	6.8
	2727	319	319	AprioriAll	71.60	418.28	6.4
				AprioriAll (Par.)	31.31	4202.50	19.7
				PrefixSpan	0.61	245.47	11.5
				SPADE	2.03	3999.67	20.1
				GSP	7.49	578.91	7.2
	3637	174	174	AprioriAll	39.64	430.87	6.4
				AprioriAll (Par.)	26.06	4203.62	14.5
				PrefixSpan	0.55	240.89	11.8
				SPADE	1.85	3898.25	20.4
				GSP	3.78	580.30	7.5
BMSWebView1	59,601	120	798	AprioriAll	1592.26	113.22	6.4
				AprioriAll (Par.)	150.97	1136.16	84.0
				PrefixSpan	0.34	129.27	15.8
				SPADE	1.00	1552.31	12.2
				GSP	205.58	2330.51	6.5

(continued on next page)

Table 7 (continued)

Dataset	Sequence Count	Minimum Support Count	Frequent Patterns	Algorithm	Execution Time (s)	Peak Memory (MB)	CPU Usage (%)
		300	201	AprioriAll	405.29	112.17	6.4
				AprioriAll (Par.)	53.25	1129.96	68.2
				PrefixSpan	0.33	121.46	16.7
				SPADE	0.56	492.45	19.1
				GSP	55.13	1522.64	6.7
		600	77	AprioriAll	67.96	110.03	6.4
				AprioriAll (Par.)	14.53	1146.86	36.3
				PrefixSpan	0.29	119.96	14.3
				SPADE	0.51	480.11	18.4
				GSP	10.90	557.98	6.8

implementations of sequential pattern mining methods. Sequence mining is a well-established technique, and this package improves its accessibility within the data science community. Moreover, by leveraging the Python programming language, the implementation is fully accessible, easy to integrate with existing source code repositories, and suitable for educational, research, and methodological applications.

First, in research settings, it can serve as a reference model for researchers seeking to compare the performance and behavior of alternative sequential pattern mining algorithms. The package provides a validated baseline implementation that facilitates benchmarking, algorithm validation, scalability studies, and methodological experimentation involving sequential data analysis. Its integration with the Python ecosystem makes it usable in common data science workflows for educational and applied research purposes.

Second, in educational environments, the package can support data mining and machine learning courses by exposing the intermediate stages of sequential pattern discovery, including candidate generation, support counting, pruning, and maximal sequence extraction.

Third, in business settings, the implementation can serve as a lightweight experimental framework for sequential data analysis tasks involving transactional and temporal datasets in domains such as cybersecurity, e-commerce, finance, healthcare, and marketing.

As empirically demonstrated in the performance evaluation, despite *AprioriAllPkg*'s transparency and interpretability, one must consider its well-known scalability limitations. The reliance on iterative candidate generation and repeated support counting becomes computationally expensive for large, dense, or high-dimensional sequence databases. Consistent with our benchmark results, more advanced architectures such as PrefixSpan [2] and SPADE [9], as well as modern parallel sequential pattern mining methods [6], achieve significantly improved runtime performance and memory efficiency in such scenarios.

Nonetheless, the primary objective of the presented package is not to introduce computational refinements to the AprioriAll algorithm. On the contrary, by preserving its original design, we provide a transparent and reproducible reference implementation.

6. Conclusions

This paper introduces a Python-based implementation of the AprioriAll sequential pattern mining algorithm, providing a practical resource for researchers and practitioners in the data science community. It facilitates understanding and development of data mining algorithms designed not only for transactional data streams but also for other time-oriented datasets.

The contribution of the *AprioriAllPkg* package is at the software level rather than the algorithmic level. The introduced implementation provides a transparent, modular, and reproducible realization of the AprioriAll workflow that enables users to inspect, reproduce, benchmark, and extend individual stages of the mining process. In addition, the package includes reproducible validation and performance evaluation workflows that facilitate correctness verification, scalability analysis, and comparative studies against other sequential pattern mining algorithms. These features make the software suitable for educational

purposes, methodological validation, benchmarking, and experimentation with sequential data mining techniques.

Although the benchmark results show that modern approaches like PrefixSpan and SPADE are more efficient than AprioriAll, the experimental results confirm the accuracy, reproducibility, and educational value of the presented implementation.

Future work includes supporting additional sequence data formats, integrating larger benchmark collections, and implementing alternative sequential pattern mining algorithms. These extensions would improve interoperability with existing data science workflows, facilitate more comprehensive performance evaluations, and enable comparative studies within a unified and reproducible software environment.

CRedit authorship contribution statement

Aleksander Kruczkowski: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Pawel Weichbroth:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The research was supported by the project “Cloud Artificial Intelligence Service Engineering (CAISE) platform to create universal and smart services for various application areas”, No. KPOD.05.10-IW.10-0005/24, as part of the European IPCEI-CIS program, financed by NRRP (National Recovery and Resilience Plan) funds.

References

- [1] Mooney C, Roddick J. Sequential pattern mining—approaches and algorithms. *ACM Comput Surv (CSUR)* 2013;45:1–39.
- [2] Pei J, Han J, Mortazavi-Asl B, Wang J, Pinto H, Chen Q, Dayal U, Hsu M. Mining sequential patterns by pattern-growth: the PrefixSpan approach. *IEEE Trans Knowl Data Eng* 2004;16:1424–40.
- [3] Ayres J, Flannick J, Gehrke J, Yiu T. Sequential pattern mining using a bitmap representation. In: Proceedings of the eighth ACM SIGKDD; 2002. p. 429–35.
- [4] Agrawal R, Srikant R. Mining sequential patterns. In: Proceedings of the eleventh international conference on data engineering; 1995. p. 3–14.
- [5] Mabroukeh N, Ezeife C. A taxonomy of sequential pattern mining algorithms. *ACM Comput Surv (CSUR)* 2010;43:1–41.
- [6] Gan W, Lin J, Fournier-Viger P, Chao H, Yu P. A survey of parallel sequential pattern mining. *ACM Trans Knowl Discov Data (TKDD)* 2019;13:1–34.
- [7] Abdullah Z, Adam O, Herawan T, Deris M. A review on sequential pattern mining algorithms based on apriori and patterns growth. In: Proceedings of the international conference on data engineering 2015 (DaEng-2015); 2019. p. 111–8.
- [8] Fournier-Viger P, Gomariz A, Gueniche T, Soltani A, Wu C, Tseng V. SPMF: a Java open-source pattern mining library. *J Mach Learn Res* 2014;15:3389–93.
- [9] Zaki M. SPADE: an efficient algorithm for mining frequent sequences. *Mach. Learn.* 2001;42:31–60.